Aula prática

Aula 7

Relacionamentos no MongoDB Embedding vs Referencing

1. Crie uma coleção estacoes_embed onde cada estação contenha:

   a) nome

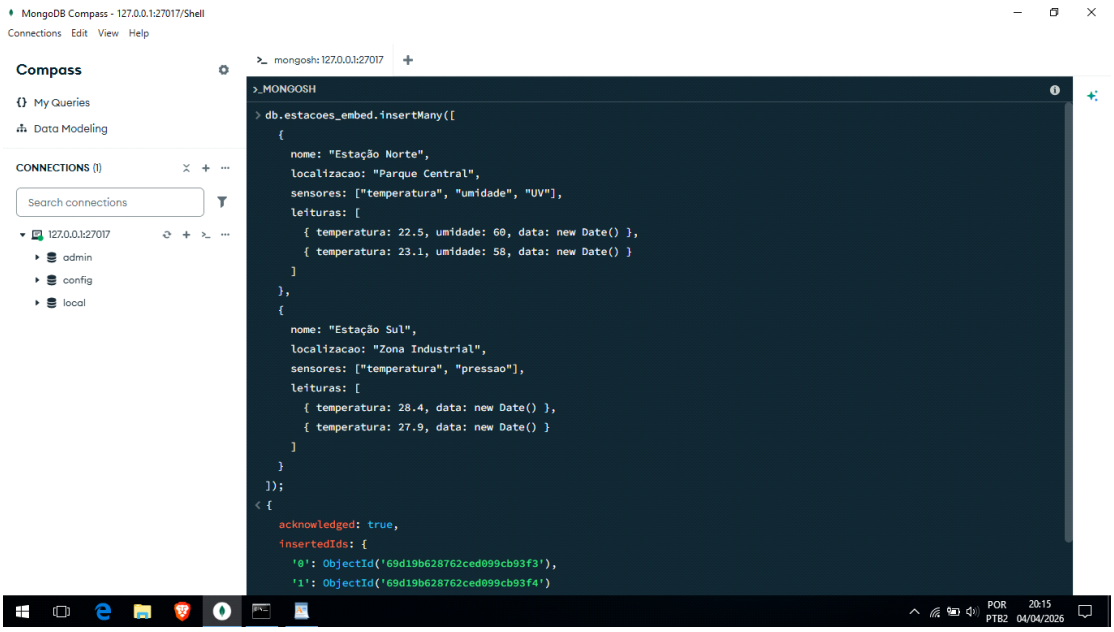
   b) localização

   c) sensores (array)

   d) leituras (array de documentos com temperatura e umidade)

   Insira 2 estações com pelo menos 2 leituras cada.

   Print:

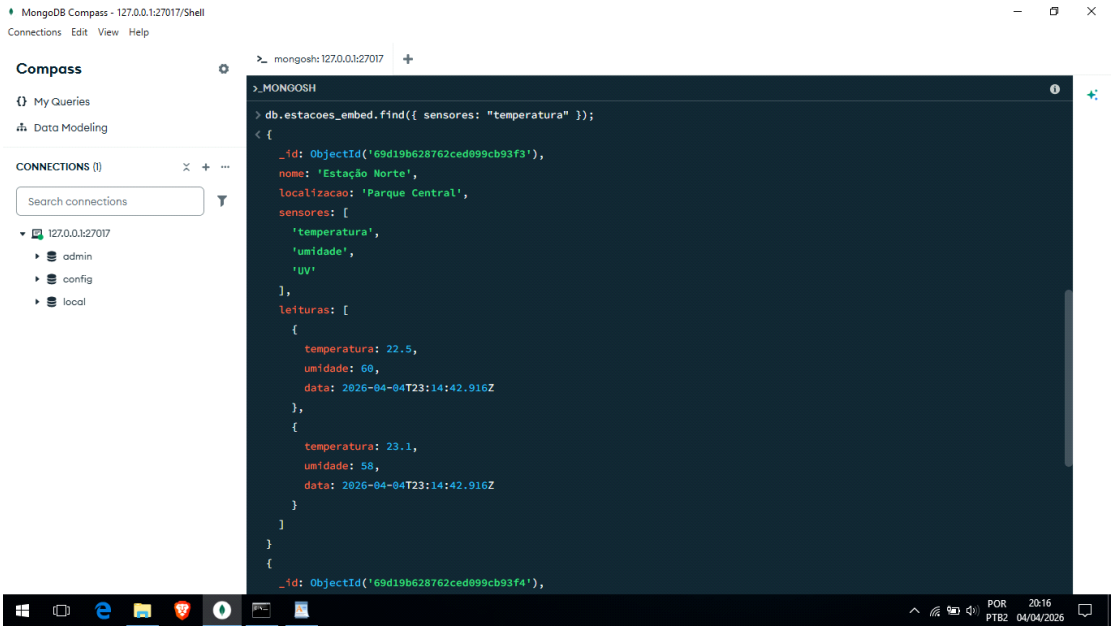


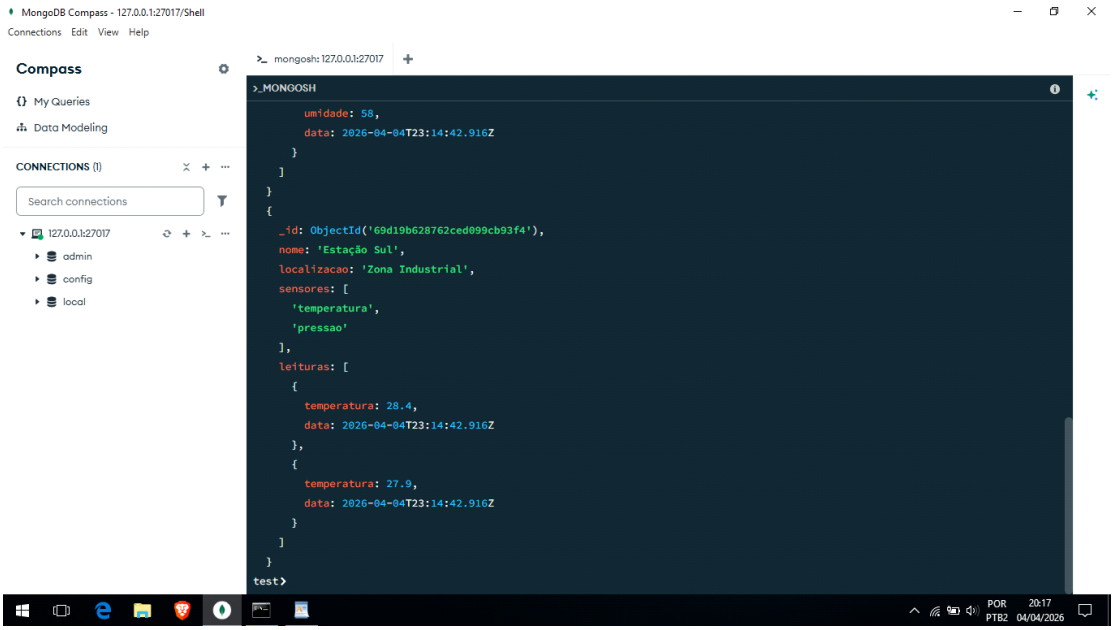
Código:

```
db.estacoes_embed.insertMany([
  {
    nome: "Estação Norte",
    localizacao: "Parque Central",
    sensores: ["temperatura", "umidade", "UV"],
    leituras: [
      { temperatura: 22.5, umidade: 60, data: new Date() },
      { temperatura: 23.1, umidade: 58, data: new Date() }
    ]
  },
  {
    nome: "Estação Sul",
    localizacao: "Zona Industrial",
    sensores: ["temperatura", "pressao"],
    leituras: [
      { temperatura: 28.4, data: new Date() },
      { temperatura: 27.9, data: new Date() }
    ]
  }
]);
```

2. Liste todas as estações que possuem o sensor temperatura.

Print:


MongoDB Compass - 127.0.0.1:27017/Shell
Connections Edit View Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
127.0.0.1:27017
admin
config
local
mongosh: 127.0.0.1:27017
>_MONGOSH
> db.estacoes_embed.find({ sensores: "temperatura" });
< {
_id: ObjectId('69d19b628762ced099cb93f3'),
nome: 'Estação Norte',
localizacao: 'Parque Central',
sensores: [
'temperatura',
'umidade',
'UV'
],
leituras: [
{
temperatura: 22.5,
umidade: 60,
data: 2026-04-04T23:14:42.916Z
},
{
temperatura: 23.1,
umidade: 58,
data: 2026-04-04T23:14:42.916Z
}
]
}
{
_id: ObjectId('69d19b628762ced099cb93f4'),
POR
PTB2
20:16
04/04/2026


MongoDB Compass - 127.0.0.1:27017/Shell
Connections Edit View Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
127.0.0.1:27017
admin
config
local
mongosh: 127.0.0.1:27017
>_MONGOSH
umidade: 58,
data: 2026-04-04T23:14:42.916Z
}
]
}
{
_id: ObjectId('69d19b628762ced099cb93f4'),
nome: 'Estação Sul',
localizacao: 'Zona Industrial',
sensores: [
'temperatura',
'pressao'
],
leituras: [
{
temperatura: 28.4,
data: 2026-04-04T23:14:42.916Z
},
{
temperatura: 27.9,
data: 2026-04-04T23:14:42.916Z
}
]
}
test>
POR
PTB2
20:17
04/04/2026

Código:

db.estacoes_embed.find({ sensores: "temperatura" });

3. Crie:

   a) coleção estacoes_ref

   b) coleção leituras_ref

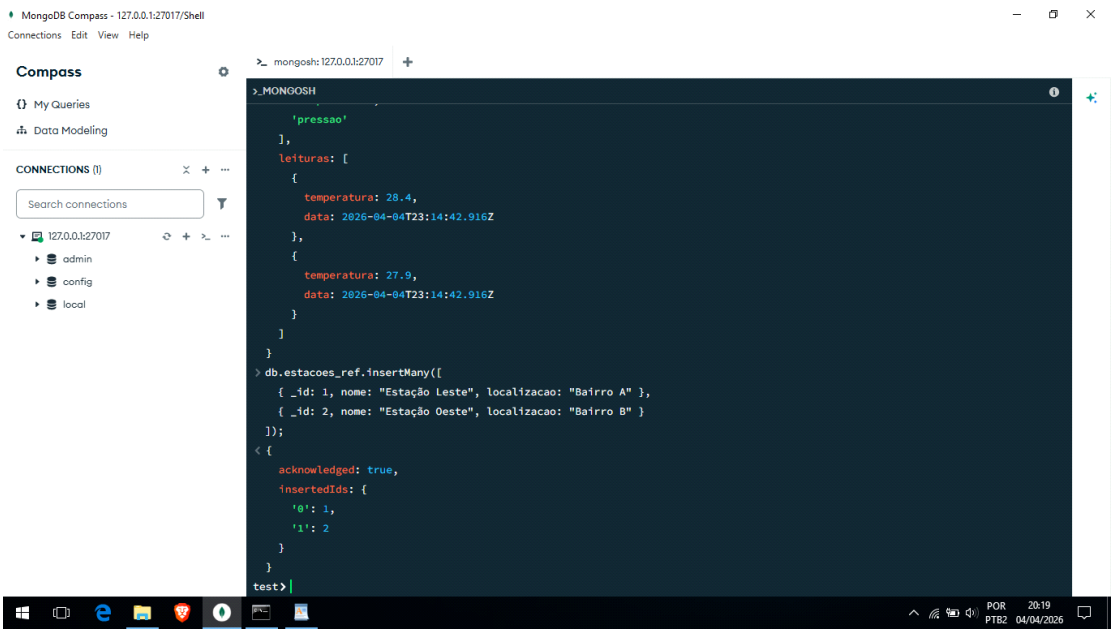
   Relacione usando estacaoId.

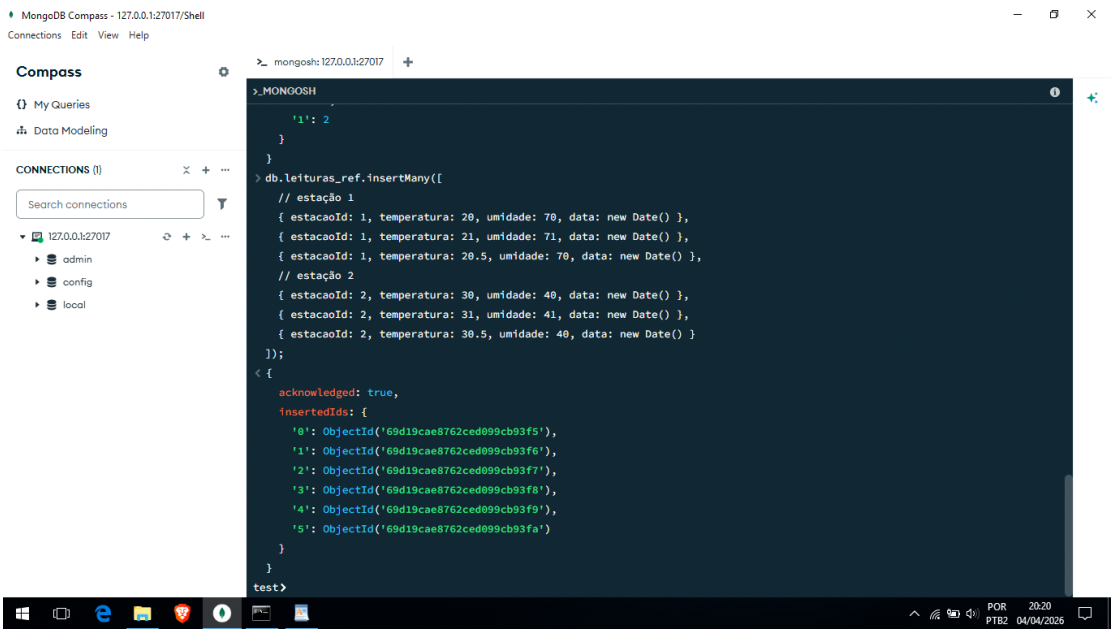
   Insira:

   a) 2 estações

   b) 3 leituras para cada estação

   Print:





Código:

```
db.estacoes_ref.insertMany([
  { _id: 1, nome: "Estação Leste", localizacao: "Bairro A" },
  { _id: 2, nome: "Estação Oeste", localizacao: "Bairro B" }
]);
db.leituras_ref.insertMany([
  // estação 1
  { estacaoId: 1, temperatura: 20, umidade: 70, data: new Date() },
  { estacaoId: 1, temperatura: 21, umidade: 71, data: new Date() },
  { estacaoId: 1, temperatura: 20.5, umidade: 70, data: new Date() },
  // estação 2
  { estacaoId: 2, temperatura: 30, umidade: 40, data: new Date() },
  { estacaoId: 2, temperatura: 31, umidade: 41, data: new Date() },
  { estacaoId: 2, temperatura: 30.5, umidade: 40, data: new Date() }
```

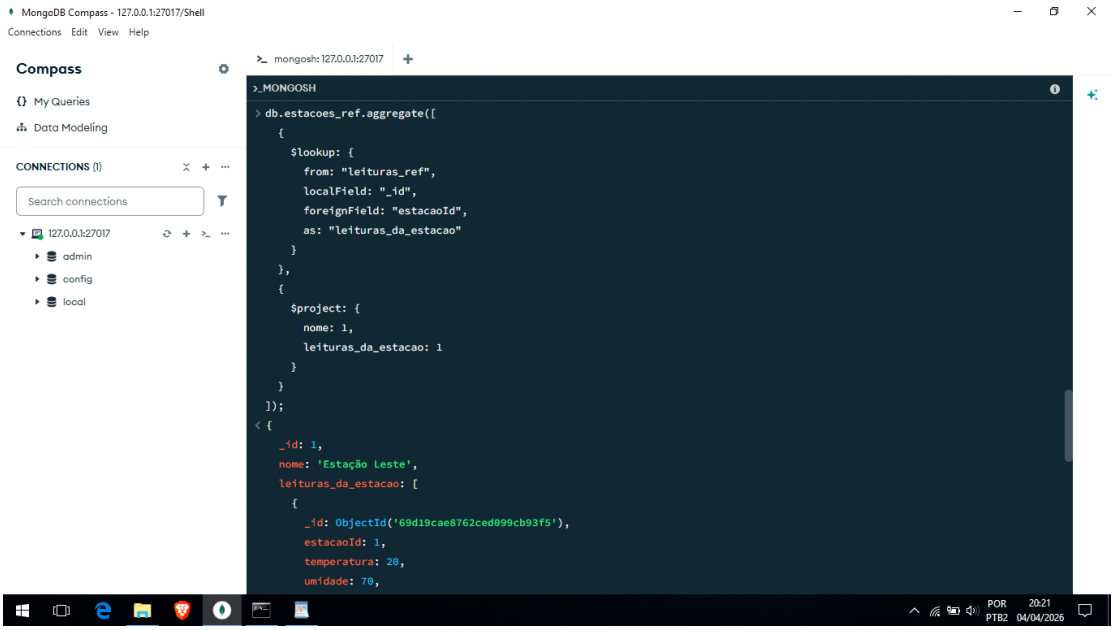
]);

4. Utilize $lookup para retornar:

   a) nome da estação

   b) suas leituras

Print:



MongoDB Compass - 127.0.0.1:27017/Shell

Connections Edit View Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

127.0.0.1:27017

admin

config

local

mongosh: 127.0.0.1:27017

>_MONGOSH

```
    },
    {
      _id: ObjectId('69d19cae8762ced099cb93f6'),
      estacaoId: 1,
      temperatura: 21,
      umidade: 71,
      data: 2026-04-04T23:20:14.009Z
    },
    {
      _id: ObjectId('69d19cae8762ced099cb93f7'),
      estacaoId: 1,
      temperatura: 20.5,
      umidade: 70,
      data: 2026-04-04T23:20:14.009Z
    }
  ]
}
{
  _id: 2,
  nome: 'Estação Oeste',
  leituras_da_estacao: [
    {
      _id: ObjectId('69d19cae8762ced099cb93f8'),
      estacaoId: 2,
      temperatura: 30,
      umidade: 40,
```

POR PTB2 20:22 04/04/2026

MongoDB Compass - 127.0.0.1:27017/Shell

Connections Edit View Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

127.0.0.1:27017

admin

config

local

mongosh: 127.0.0.1:27017

>_MONGOSH

```
  leituras_da_estacao: [
    {
      _id: ObjectId('69d19cae8762ced099cb93f8'),
      estacaoId: 2,
      temperatura: 30,
      umidade: 40,
      data: 2026-04-04T23:20:14.009Z
    },
    {
      _id: ObjectId('69d19cae8762ced099cb93f9'),
      estacaoId: 2,
      temperatura: 31,
      umidade: 41,
      data: 2026-04-04T23:20:14.009Z
    },
    {
      _id: ObjectId('69d19cae8762ced099cb93fa'),
      estacaoId: 2,
      temperatura: 30.5,
      umidade: 40,
      data: 2026-04-04T23:20:14.009Z
    }
  ]
}
test>
```

POR PTB2 20:22 04/04/2026

Código:

```
db.estacoes_ref.aggregate([
  {
    $lookup: {
      from: "leituras_ref",
      localField: "_id",
      foreignField: "estacaoId",
      as: "leituras_da_estacao"
    }
  },
  {
    $project: {
      nome: 1,
      leituras_da_estacao: 1
    }
  }
]);
```

5.

2. Qual modelo é mais rápido para leitura? Por quê?

R: O modelo embutido (embedded). Ele permite recuperar todos os dados relacionados em só uma única operação, sem precisar fazer "joins" (lookups) no servidor.

b) Qual modelo é mais escalável?

R: O modelo de referência (referenced). Já que o Mongo tem um limite de 16 MB por documento, logo se você inserir milhões de leituras em uma estação única, esse documento vai exceder o limite e causar erros de escrita, além de que documentos grandes em excesso pioram a performance.

c) Em um sistema com milhões de leituras, qual você escolheria?

R: O modelo de referência.

d) Justifique tecnicamente sua escolha

R: Graças ao limite de 16MB, milhões de leituras ultrapassam essa marca facilmente; no modelo embutido sempre que uma leitura é adicionada, o documento cresce como consequência, se precisar ser realocado no disco com frequência, pode causar sobrecarga de escrita; e com coleções separadas é possível criar índices específicos para a coleção de leituras (por data, por exemplo) sem ter que carrear os dados da estação.